Gradient-Matching Coreset Selection for Data-Efficient Training: A Reproduction and Runtime Break-Even Analysis of CRAIG

Harsh Joshi Mayank Srivastava Zayed Alam Himesh

***Abstract*—CRAIG selects a weighted subset (coreset) of the training data whose weighted gradient sum approximates the full-data gradient, by greedily maximizing a submodular facility-location objective over pairwise per-sample gradient distances [1]. We re-implement CRAIG from scratch in PyTorch [2] and evaluate it on a CPU-only machine. We account for the full wall-clock cost: per-sample gradients, pairwise similarities, greedy selection, weight assignment, and training on the subset. Unlike the original deep-network protocol, our selection is performed once, at the initial weights, with no per-epoch re-selection. On MNIST with logistic regression (5 seeds), weighted CRAIG has a higher mean accuracy than uniform random subsets at every subset ratio from 5% to 50%, e.g., $90.19\pm0.15\%$ vs. $89.68\pm0.18\%$ at 5% (full data: $92.22 \pm 0.12\%$). Without its weights, CRAIG is worse than random. However, selection takes 29.41–38.20 s against 6.58 s for full training, so the end-to-end speedup is only 0.16–0.22×: CRAIG is *slower* than full training. The lazy-greedy step accounts for 77–81% of the selection time. A dataset-size scaling study places the break-even point between N=5,000 and N=10,000 samples in our setup. On a reduced CIFAR-10 setting (10k training images, 3 seeds), CRAIG has a much lower gradient approximation error than random sampling (0.207 vs. 0.802 at 10%). Yet it beats random in accuracy only at the 10% ratio. On synthetic time series, accuracy results are too unstable to support a conclusion. We report these negative results explicitly and discuss when gradient-matching coresets can pay off.**

***Index Terms*—coresets, data subset selection, submodular optimization, facility location, data-efficient training, reproducibility**

I. Introduction

Training cost grows with the size of the training set. One way to reduce it is to train on a small, carefully chosen subset. Mirzasoleiman, Bilmes, and Leskovec proposed CRAIG (*Coresets for Accelerating Incremental Gradient descent*) [1]. CRAIG selects a subset S and per-element weights such that the weighted subset gradient approximates the full gradient. The selection problem is cast as maximizing a monotone submodular facility-location function. Greedy maximization of such a function enjoys a $(1-1/e)$ approximation guarantee [3].

A coreset method helps in practice only if the time spent *selecting* the subset, plus the time spent training on it, is less than the time for training on the full data. This report is a reproduction of CRAIG. It is designed around that end-to-end accounting and run on commodity CPU hardware. Our contributions are:

- A from-scratch, tested implementation of the CRAIG pipeline. It computes exact per-sample gradients with `torch.func` (`vmap` over `grad`), builds blockwise gradient-distance matrices, runs naive and lazy [4] greedy facility location, and assigns Voronoi weights $w_j = |C_j|$.
- A benchmark on MNIST [5] (logistic regression, full 60k training set, 5 seeds), a reduced-scale CIFAR-10 [6] experiment, and synthetic time-series data. It compares weighted CRAIG, unweighted CRAIG, uniform random subsets, and full-data training.
- A per-stage runtime breakdown and a break-even analysis, including scaling with dataset size and sequence length.

Our main finding is mixed. The *statistical* part of CRAIG reproduces on MNIST: weighted coresets reach higher mean accuracy than random subsets of equal size, and the weights are essential. The *computational* claim does not hold in our setting: with a pure-Python lazy greedy on CPU, selection is 4.47–5.80× more expensive than full training. All experiments were run on a CPU-only machine. Several experiments are at reduced scale, as detailed in Section V.

II. Related Work

Submodular maximization. Nemhauser, Wolsey, and Fisher [3] showed that the greedy algorithm achieves a $(1-1/e)$ approximation for maximizing a monotone submodular set function under a cardinality constraint. Minoux [4] introduced the *lazy* (accelerated) greedy algorithm. It exploits the diminishing-returns property: marginal gains computed in earlier iterations remain valid upper bounds. As a result, most candidates need not be re-evaluated, and the selected set is identical to the naive greedy one, up to ties.

Coresets for training. CRAIG [1] connects these results to incremental gradient methods. It bounds the error between the full gradient and a weighted subset gradient by a facility-location objective over per-sample gradient differences. It then selects the subset greedily and trains on it with weighted incremental gradient updates. For convex problems, the original work bounds gradient differences by feature-space distances, which do not depend on the parameters. For deep networks, it uses an approximation based on last-layer gradients and re-selects the subset during training. Our work is a reproduction of this method only. We do not compare against other subset-selection families, such as importance sampling or active learning, and we make no claims about them.

Tooling. Our implementation relies on PyTorch [2], including its functional transforms (`torch.func`) for vectorized per-sample gradients.

III. Background: CRAIG

Let $V = \{(x_i, y_i)\}_{i=1}^N$ be the training set and $\ell_i(\theta)$ the loss of sample i under parameters $\theta \in \mathbb{R}^D$. Let $g_i(\theta) = \nabla_\theta \ell_i(\theta)$ be its per-sample gradient. The full gradient of the empirical risk is $G_V(\theta) = \frac{1}{N}\sum_{i\in V} g_i(\theta)$. A weighted coreset $S \subseteq V$, $|S| = k \ll N$, with weights $w_j \geq 0$, gives the approximation $G_S(\theta) = \frac{1}{N}\sum_{j\in S} w_j g_j(\theta)$.

Gradient matching. Partition V into clusters $\{C_j\}_{j\in S}$, one per representative, and set $w_j = |C_j|$, so that $\sum_j w_j = N$. By the triangle inequality,

$$\|G_V - G_S\| \leq \frac{1}{N}\sum_{j\in S}\sum_{i\in C_j} \|g_i - g_j\|. \tag{1}$$

The bound is minimized by assigning each i to its closest representative. Selecting S then becomes a k-medoids problem, $\min_{|S|\leq k}\sum_{i\in V}\min_{j\in S} d(i,j)$, with $d(i,j) = \|g_i - g_j\|$.

Facility location. With $M \geq \max_{i,j} d(i,j)$ and similarity $s(i,j) = M - d(i,j) \geq 0$, minimizing (1) is equivalent to maximizing

$$F(S) = \sum_{i\in V}\max_{j\in S} s(i,j). \tag{2}$$

F is monotone and submodular. Greedy selection, which repeatedly adds $j^\star = \arg\max_c F(S\cup\{c\}) - F(S)$, therefore achieves at least $(1 - 1/e)$ of the optimal value [3]. After selection, every i is assigned to $\arg\max_{j\in S} s(i,j)$ (a Voronoi assignment), and $w_j = |C_j|$.

Evaluation quantities. We measure the gradient approximation quality at the parameters used for selection by the relative error $\|\sum_i g_i - \sum_{j\in S} w_j g_j\| / \|\sum_i g_i\|$ and by the cosine similarity between the two sums. For random or unweighted subsets we use the uniform weights $w_j = N/k$. The run time of CRAIG is $T_{\text{CRAIG}} = T_{\text{sel}} + T_{\text{sub}}$, where T_{sel} includes gradients, similarity, greedy selection, and weight assignment. We report the training-only speedup $T_{\text{full}}/T_{\text{sub}}$ and the end-to-end speedup $T_{\text{full}}/(T_{\text{sel}} + T_{\text{sub}})$. A true computational win requires $T_{\text{sel}} < T_{\text{full}} - T_{\text{sub}}$.

IV. Method and Implementation

The implementation lives in the `craig/` package of the project repository. A unit-test suite covers gradients, similarity, facility location, the selector, and training; all 23 tests pass.

Per-sample gradients. The module `craig/gradients.py` computes the $N \times D$ gradient matrix. It applies `torch.func.vmap` over `torch.func.grad` of a single-sample loss evaluated through `functional_call`, processing chunks of 64–128 samples to bound memory. If vectorization fails, the code falls back to one backward pass per sample; it never reuses a batch gradient for individual samples. Gradients are computed with the model in evaluation mode, so BatchNorm layers use their running statistics.

Representations. The module `craig/representations.py` supports three representations: the full parameter gradient; the gradient of the final linear (classifier) layer only; and an optional Johnson–Lindenstrauss random projection (Gaussian or Rademacher). The projection was not used in the reported experiments. We use the full gradient for logistic regression ($D = 7{,}850$) and for the 1D time-series CNN ($D = 23{,}492$). For the CIFAR-10 CNN we use last-layer gradients ($D = 1{,}290$).

Algorithm 1 Lazy greedy facility location (per class)

Require: similarity matrix $s \in \mathbb{R}_{\geq 0}^{n\times n}$, budget k
$c \leftarrow \mathbf{0}_n$ {current coverage $\max_{j\in S} s(i,j)$}
push $(\sum_i s(i,j),\ j)$ for all j into max-heap H; $S \leftarrow \emptyset$
while $|S| < k$ **do**
  pop $(\cdot, j)$ from H; $\delta \leftarrow \sum_i \max(0, s(i,j) - c_i)$
  if $H = \emptyset$ **or** $\delta \geq$ top bound of H **then**
    $S \leftarrow S \cup \{j\}$; $c \leftarrow \max(c, s(\cdot, j))$
  else
    push (δ, j) back into H
  end if
end while
$w_j \leftarrow |\{i : \arg\max_{j'\in S} s(i,j') = j\}|$ for $j \in S$
return S, w

Similarity. The module `craig/similarity.py` computes Euclidean (or cosine) distances blockwise, using $\|x\|^2 + \|y\|^2 - 2x^\top y$ on row blocks of 1024. A 4 GiB guard limits the size of the dense matrix. Similarities are $s = \max(D) - D$, where the maximum is taken within the matrix being built.

Greedy facility location. The module `craig/facility_location.py` provides a naive vectorized greedy algorithm and a lazy greedy algorithm (Alg. 1) [4]. The lazy variant keeps a max-heap of stale marginal gains. It recomputes the gain of the top candidate and accepts it if it is still at least the next-best upper bound. The lazy variant was used in all experiments.

CraigSelector. `craig/selector.py` runs the pipeline gradients → similarity → greedy → weights. Selection is done *per class*: each class c with n_c samples receives a budget of $\text{round}(k\, n_c/N)$. This keeps the dense matrices at size $n_c \times n_c$. Because of the rounding, the realized CRAIG subset size can differ from k by a few samples; for example, it is 5,999 instead of 6,000 at 10% on MNIST. Every stage is timed with `time.perf_counter`, with device synchronization if a GPU is used.

Weighted training. When weights are used, the minibatch loss is $\sum_{i\in B} w_i \ell_i / \sum_{i\in B} w_i$. This is a per-minibatch normalized weighted loss. It is a simplification of the weighted incremental-gradient step sizes described in [1]. CRAIG-unweighted trains on the same subset with the standard mean loss.

V. Experimental Setup

Hardware and software. All runs used a single Linux machine with 8 CPU cores (Intel Xeon) and a CPU-only PyTorch build (2.15.0+cpu). No GPU or Apple-MPS runs were performed. Device-selection code paths for those backends

exist but are untested. All times are wall-clock seconds. The machine was not isolated, so the timings contain noise: for example, the standard deviation of full CIFAR-10 training time across seeds is 3.90 s on a mean of 13.17 s.

Methods. We compare four methods. FULL trains on all N samples. RANDOM trains on a uniform random subset of size k, with unweighted loss. CRAIG trains on the CRAIG subset with weights $w_j = |C_j|$. CRAIG-unweighted trains on the same subset as CRAIG but without weights. All methods share the optimizer, learning rate, batch size, number of epochs, and model initialization for a given seed. T_{sel} is recomputed for every (seed, ratio) pair; nothing is cached. Test-set evaluation is excluded from the training time.

Selection protocol. CRAIG selects *once*, using gradients at the seed's initial weights. The same initialization is then trained on the subset. This is a deliberate simplification. For deep networks, the original method re-selects the subset during training [1]. For convex models, it uses a parameter-independent feature-space bound instead of exact gradients at a particular θ.

MNIST [5]. We train multinomial logistic regression ($784 \to 10$) on the full 60k training set and evaluate on the 10k test set. Inputs are normalized. We use SGD (lr 0.01, momentum 0.9, weight decay 10^{-4}), batch size 128, and 20 epochs. Seeds are $\{42, 123, 456, 789, 1000\}$ and subset ratios are $\{5, 10, 20, 30, 50\}\%$. CRAIG uses full gradients and Euclidean distance.

CIFAR-10 [6] *(reduced scale).* We use only the first 10,000 training images and the full 10k test set, with per-channel normalization and no augmentation. The model is a small CNN with two 3×3 conv layers (32 and 64 channels, each followed by max-pooling) and fully connected layers $4096 \to 128 \to 10$, for 545,098 parameters in total. We use SGD (lr 0.01, momentum 0.9, weight decay 5×10^{-4}), batch size 128, and 10 epochs. Seeds are $\{42, 123, 456\}$ and ratios are $\{10, 20, 30, 50\}\%$. CRAIG uses last-layer gradients. This setting is far from the original paper's CIFAR-10 setup, and FULL reaches only 59.02%.

Synthetic time series. No real time-series dataset (e.g., from the UCR archive) was available, so we use a synthetic generator. A sample of class $c \in \{0, 1, 2, 3\}$ is $\sin(2\pi f_c t + \phi) + \varepsilon$, with $f_c = 2(c+1)$, a random phase ϕ, and Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Each sequence has $L = 100$ points, the training set has $N = 4{,}000$ sequences, and the test set has 1,000. The model is a 1D CNN with three conv+BatchNorm layers, global average pooling, and a linear head. We train with Adam (lr 10^{-3}, weight decay 10^{-4}), batch size 64, for 10 epochs, with 3 seeds and the same five ratios as MNIST. We run an *easy* setting (σ=0.2) and a *hard* setting (σ=2.0).

Scaling studies (2 seeds $\{42, 123\}$, ratio 10%, 10 epochs, FULL/RANDOM/CRAIG only). In the *dataset-size* study, we train MNIST logistic regression on the first $N \in \{1{,}000, \ldots, 20{,}000\}$ training images, with the same optimizer as above. In the *sequence-length* study, we use the synthetic time series (σ=0.2) with N=2,000 and $L \in \{50, 100, 200, 500, 1000\}$.

TABLE I
MNIST LOGISTIC REGRESSION: TEST ACCURACY (%), MEAN ± STD OVER 5 SEEDS. k IS THE RANDOM SUBSET SIZE; CRAIG'S PER-CLASS ROUNDING GIVES $k \pm 1$.

Ratio	k	CRAIG	CRAIG-unw.	RANDOM
5%	3000	90.19 ± 0.15	89.53 ± 0.21	89.68 ± 0.18
10%	6000	91.07 ± 0.11	89.81 ± 0.28	90.45 ± 0.21
20%	12000	91.59 ± 0.16	89.98 ± 0.49	91.15 ± 0.34
30%	18000	91.71 ± 0.19	90.59 ± 0.24	91.51 ± 0.29
50%	30000	92.14 ± 0.18	91.57 ± 0.31	91.89 ± 0.14
100% (FULL)	60000		92.22 ± 0.12	

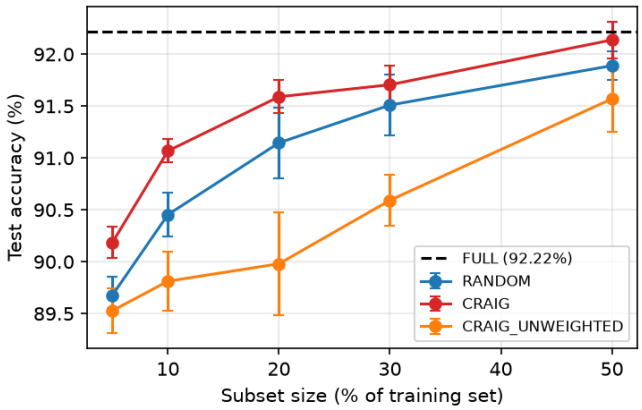


Fig. 1. MNIST: test accuracy vs. subset size (mean ± std, 5 seeds). The dashed line is FULL (92.22%).

All numbers below are computed directly from the experiment CSV files (`results/raw`, `results/processed`) by a script, `paper/make_tables.py`, that emits the table bodies. Means ± sample standard deviations are taken over seeds. Speedups are ratios of mean times.

VI. RESULTS

A. MNIST: accuracy

Table I and Fig. 1 show that weighted CRAIG has the highest mean accuracy among the subset methods at every ratio. At 50% it comes within 0.08 points of FULL ($92.14 \pm 0.18\%$ vs. $92.22 \pm 0.12\%$). Because all methods share the seed, we also report paired per-seed differences (Table II). CRAIG beats RANDOM in 5/5 seeds at 5–20% (by $+0.44$ to $+0.61$ points on average). At 30% and 50% it wins in only 4/5 seeds, and the mean gaps ($+0.20$, $+0.25$) are comparable to the seed-to-seed spread. The weights matter: CRAIG-unweighted trains on exactly the same points but is *below* RANDOM at every ratio. Weighted CRAIG beats it in 5/5 seeds at every ratio, by $+0.57$ to $+1.61$ points.

B. MNIST: gradient approximation error

On MNIST, the gradient error of weighted CRAIG is only marginally below that of RANDOM (Table III): 0.118 vs. 0.128 at 5% and 0.027 vs. 0.030 at 50%. Its seed-to-seed spread is several times smaller. The unweighted CRAIG estimate (same points, uniform weights N/k) behaves very

TABLE II
PAIRED PER-SEED ACCURACY DIFFERENCE OF WEIGHTED CRAIG VS. THE BASELINE (PERCENTAGE POINTS, MEAN ± STD; IN PARENTHESES, THE NUMBER OF SEEDS WHERE CRAIG IS HIGHER). †REDUCED SCALE.

Data	Ratio	CRAIG−RANDOM	CRAIG−CRAIG-unw.
MNIST	5%	$+0.51 \pm 0.28$ (5/5)	$+0.66 \pm 0.25$ (5/5)
	10%	$+0.61 \pm 0.25$ (5/5)	$+1.26 \pm 0.22$ (5/5)
	20%	$+0.44 \pm 0.46$ (5/5)	$+1.61 \pm 0.45$ (5/5)
	30%	$+0.20 \pm 0.33$ (4/5)	$+1.11 \pm 0.34$ (5/5)
	50%	$+0.25 \pm 0.19$ (4/5)	$+0.57 \pm 0.20$ (5/5)
CIFAR-10†	10%	$+0.87 \pm 0.17$ (3/3)	$+2.00 \pm 1.55$ (3/3)
	20%	-0.82 ± 0.37 (0/3)	-0.19 ± 1.12 (2/3)
	30%	-2.38 ± 1.45 (0/3)	-2.30 ± 2.54 (0/3)
	50%	-1.54 ± 2.83 (1/3)	-1.38 ± 1.18 (0/3)
TS (σ=2)	5%	-0.80 ± 8.94 (2/3)	-0.93 ± 3.10 (1/3)
	10%	-4.80 ± 1.51 (0/3)	-2.63 ± 2.29 (0/3)
	20%	$+1.13 \pm 4.02$ (2/3)	-6.30 ± 1.85 (0/3)
	30%	-5.23 ± 18.97 (1/3)	-5.60 ± 0.95 (0/3)
	50%	-7.17 ± 15.57 (1/3)	-0.97 ± 17.00 (2/3)

TABLE III
MNIST: RELATIVE GRADIENT ERROR AT THE INITIAL WEIGHTS (MEAN ± STD, 5 SEEDS) AND MEAN COSINE SIMILARITY BETWEEN $\sum_i g_i$ AND THE SUBSET ESTIMATE.

	Relative error			Cosine	
Ratio	CRAIG	CRAIG-unw.	RANDOM	CRAIG	RANDOM
5%	0.118 ± 0.003	0.110 ± 0.006	0.128 ± 0.013	0.9956	0.9919
10%	0.087 ± 0.002	0.126 ± 0.008	0.090 ± 0.011	0.9975	0.9959
20%	0.061 ± 0.002	0.185 ± 0.006	0.063 ± 0.010	0.9987	0.9980
30%	0.047 ± 0.001	0.203 ± 0.008	0.048 ± 0.007	0.9992	0.9988
50%	0.027 ± 0.001	0.173 ± 0.007	0.030 ± 0.004	0.9997	0.9995

differently. Its error *grows* from 0.110 at 5% to 0.203 at 30%. This shows that the medoids alone are a biased sample of the gradient distribution, and that the Voronoi weights $w_j = |C_j|$ correct the bias.

C. MNIST: time breakdown and break-even

Table IV and Figs. 3 and 4 give the main negative result. Training on the coreset is 1.58–14.29× faster than full training. Selection, however, takes 29.41–38.20 s, which is 4.47–5.80× the 6.58 s full-training time. The end-to-end speedup is therefore 0.16–0.22×, and CRAIG does *not* break even at any ratio on MNIST. RANDOM, whose selection costs below 2 ms, has end-to-end speedups of 1.98–19.62×.

The lazy greedy step dominates selection: it takes 22.71–29.33 s, or 77–81% of T_{sel}. Similarity computation takes 13–18%, and the vectorized per-sample gradients only about 1 s (2.7–3.7%). Selection time also varies substantially across seeds (std 3.88–7.17 s).

D. CIFAR-10 (reduced scale)

On reduced CIFAR-10 (Tables VI and VII, Figs. 2b and 6), last-layer CRAIG approximates the last-layer gradient far better than random sampling: the relative error is 0.207 vs. 0.802 at 10% and 0.095 vs. 0.254 at 50%. This does *not* translate into accuracy. CRAIG beats RANDOM only at 10% ($37.00 \pm 1.58\%$ vs. $36.13 \pm 1.71\%$; 3/3 seeds). It is lower at 20%, 30%, and 50%, by 0.82 to 2.38 points on average (Table II). All subset accuracies remain 7.3–24.0 points below FULL ($59.02 \pm 0.17\%$). Selection is cheap here, about 1 s, because $D = 1{,}290$ and each class has only about 1,000 samples. CRAIG therefore breaks even at every ratio (end-to-end speedup 1.75–5.07×). RANDOM is still faster at matching or higher accuracy. We attribute the accuracy result mainly to the one-shot selection at a random initialization. At that point, last-layer gradients describe an untrained network, so a subset that matches them need not stay representative as the features change.

E. Synthetic time series

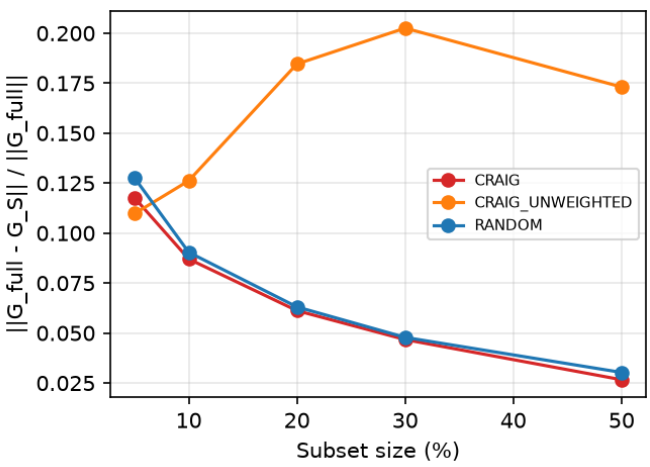
In the easy setting (σ=0.2), every method reaches 100% test accuracy at every ratio and seed. The task therefore cannot separate the methods. Table VIII and Fig. 7 report the hard setting (σ=2.0). Here the results are unstable. FULL reaches only $52.87 \pm 5.90\%$, below several subset configurations. At 30% and 50%, the standard deviations reach 11.9–16.5 points. Weighted CRAIG has a higher mean than RANDOM only at 20%. No reliable accuracy conclusion can be drawn; the training setup needs tuning, and BatchNorm instability between training and evaluation is one plausible cause. The gradient-matching objective itself still behaves as expected. CRAIG's relative gradient error is roughly an order of magnitude below RANDOM's (e.g., 0.070 vs. 0.662 at 10%). Selection costs about 0.7 s against 2.52 s for full training, so CRAIG breaks even at every ratio (1.38–2.92×).

F. Scaling with dataset size

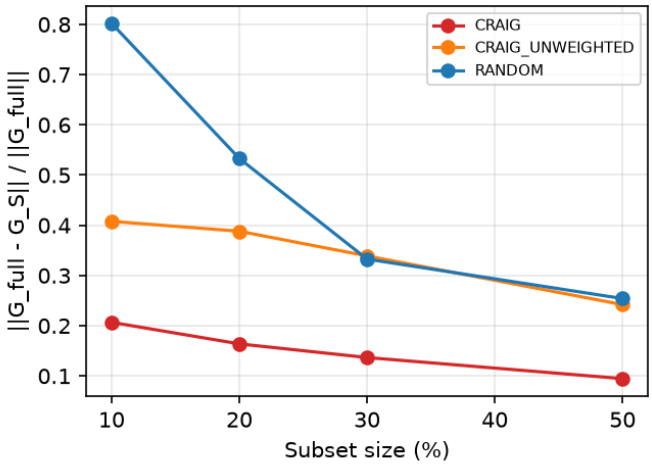
Fig. 5a and the top of Table V show that selection cost grows faster than full training. From N=1,000 to 20,000 (a 20× increase), T_{full} grows by 20.4×, T_{sel} by 117.8×, and the facility-location time by 368.4×. From 10,000 to 20,000 alone, the facility-location time grows by 7.29×, consistent with the quadratic size of the per-class similarity matrices and the cost of the greedy passes over them. The end-to-end speedup falls from 1.50× at N=1,000 to 1.12× at 5,000, then to 0.75× at 10,000 and 0.33× at 20,000. *The break-even point therefore lies between N=5,000 and N=10,000 in this setup.* At every size, CRAIG has the higher mean accuracy at the 10% ratio (e.g., 76.66% vs. 72.83% at N=1,000 and 89.54% vs. 89.05% at 20,000). The gap shrinks as N grows. Only two seeds were run, so these gaps carry limited statistical weight.

G. Scaling with sequence length

The 1D CNN ends in global average pooling, so its parameter count, and hence the gradient dimension D, does not depend on L. As a result, the similarity and facility-location times stay roughly flat across L (bottom of Table V, Fig. 5b; the dip in similarity time at L=1000 was not investigated). Only the gradient computation grows: from 0.126 s at L=50 to 1.499 s at L=1000, an 11.9× increase. Full training also grows with L, from 1.058 s to 5.328 s, so CRAIG breaks even at every length (2.25–3.01×). In accuracy, however, CRAIG is below RANDOM for $L \geq 200$ (e.g., 54.60% vs. 64.25% at



(a) MNIST (5 seeds)



(b) CIFAR-10, reduced scale (3 seeds, last-layer gradients)

Fig. 2. Relative gradient approximation error $\|G_V - G_S\|/\|G_V\|$ at the initial weights vs. subset size. Weighted CRAIG has a lower error than RANDOM at every ratio. The gap is small on MNIST and large on CIFAR-10. On MNIST at 5%, the unweighted estimate is slightly lower (0.110 vs. 0.118).

TABLE IV
MNIST: MEAN CRAIG TIME PER STAGE (S, 5 SEEDS). "WEIGHTS" = T_{sel} − (GRADIENTS + SIMILARITY + GREEDY) IS THE VORONOI WEIGHT-ASSIGNMENT TIME. SPEEDUPS ARE RELATIVE TO FULL TRAINING ($T_{\text{full}} = 6.58$ S).

Ratio	Gradients	Similarity	Lazy greedy	Weights	T_{sel} (mean±std)	T_{sub}	$T_{\text{sel}} + T_{\text{sub}}$	Train-only speedup	End-to-end speedup
5%	1.07	5.37	22.71	0.25	29.41 ± 6.30	0.46	29.87	14.29	0.22
10%	1.01	4.52	25.85	0.43	31.81 ± 3.88	0.84	32.65	7.80	0.20
20%	1.05	4.84	27.31	0.84	34.04 ± 4.87	1.71	35.75	3.85	0.18
30%	1.02	4.47	28.70	1.19	35.38 ± 6.84	2.61	37.99	2.52	0.17
50%	1.04	5.27	29.33	2.56	38.20 ± 7.17	4.16	42.36	1.58	0.16
FULL	–	–	–	–	–	6.58 ± 0.60	6.59	1.00	1.00

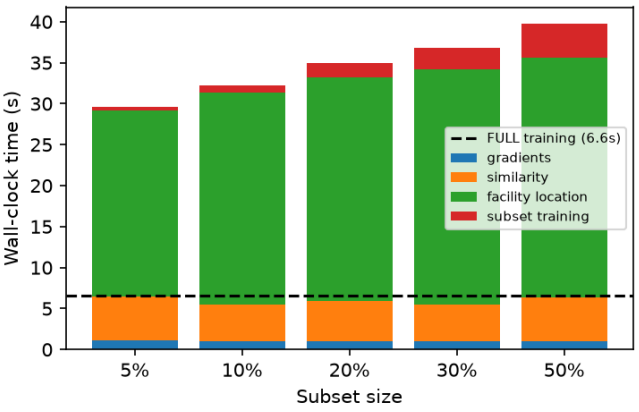


Fig. 3. MNIST: mean CRAIG wall-clock time by stage (5 seeds). The dashed line is FULL training (6.6 s). The weight-assignment component of Table IV is not drawn.

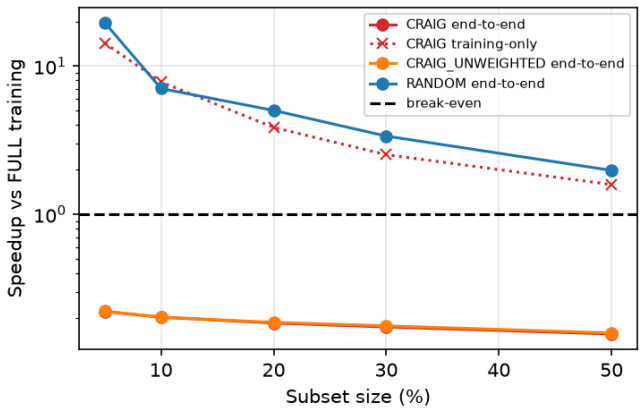
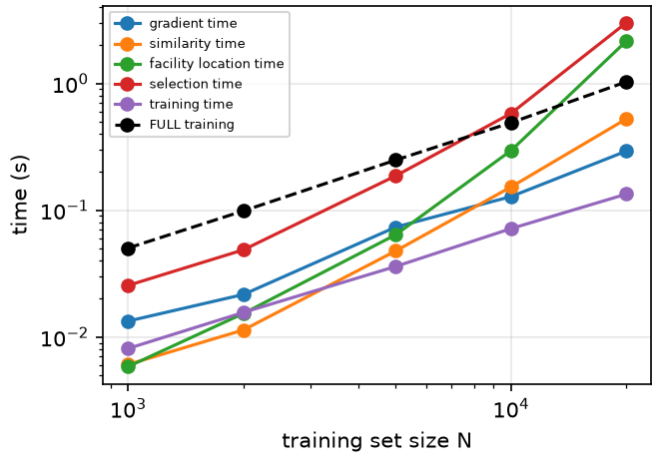


Fig. 4. MNIST: speedup relative to FULL training (log scale). CRAIG's training-only speedup is above 1, but its end-to-end speedup (selection included) is 0.16–0.22×, below the break-even line. The curves for weighted and unweighted CRAIG overlap.

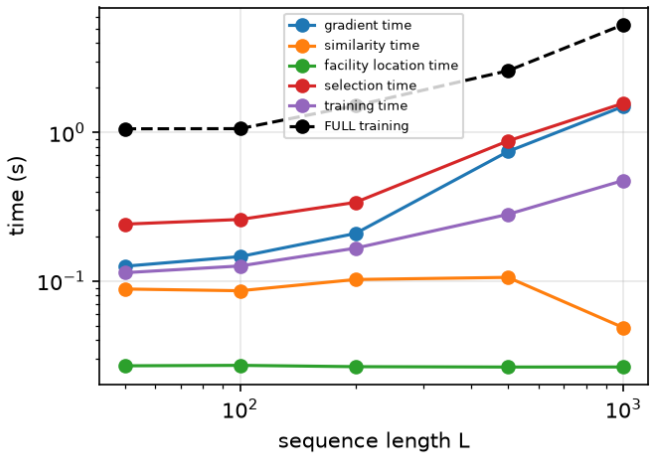
L=1000). The variances are large, and FULL itself drops to $84.35 \pm 10.39\%$ at L=1000.

VII. DISCUSSION

The weights matter. The most robust result of this reproduction is the role of the Voronoi weights. On MNIST, the same CRAIG points used without weights give a larger gradient error than random sampling (Table III) and lower accuracy at every ratio. On MNIST, weighted CRAIG beats unweighted CRAIG in every seed at every ratio (Table II). The medoids returned by facility location are, by construction, not a uniform sample. Treating them as one discards the information about how much of the data each medoid represents.



(a) MNIST logistic regression, training-set size N



(b) Synthetic time series (N=2,000), sequence length L

Fig. 5. Scaling of CRAIG's cost (ratio 10%, 10 epochs, mean over 2 seeds, log–log axes). The curves show the CRAIG stage times, CRAIG subset training ("training time"), and FULL training (dashed). In (a), selection overtakes FULL training between N=5,000 and N=10,000. In (b), only the gradient time grows with L. Accuracies are given in Table V.

TABLE V
SCALING STUDIES (RATIO 10%, 10 EPOCHS, 2 SEEDS). TOP: MNIST LOGISTIC REGRESSION WITH TRAINING-SET SIZE N (MEAN ACCURACIES). BOTTOM: SYNTHETIC TIME SERIES (σ=0.2, N=2000) WITH SEQUENCE LENGTH L (ACCURACIES AS MEAN ± STD). TIMES ARE MEAN SECONDS; ACCURACIES ARE IN %.

N	T_{grad}	T_{sim}	T_{FL}	T_{sel}	T_{full}	CRAIG end-to-end speedup	CRAIG acc.	RANDOM acc.	FULL acc.
1,000	0.013	0.006	0.006	0.026	0.050	1.50	76.66	72.83	86.50
2,000	0.022	0.011	0.015	0.049	0.099	1.54	82.31	79.16	88.50
5,000	0.074	0.048	0.064	0.187	0.249	1.12	86.65	85.11	90.06
10,000	0.128	0.153	0.296	0.580	0.490	0.75	88.67	87.76	90.98
20,000	0.292	0.525	2.155	3.009	1.029	0.33	89.54	89.05	91.51
L	T_{grad}	T_{sim}	T_{FL}	T_{sel}	T_{full}	CRAIG end-to-end speedup	CRAIG acc.	RANDOM acc.	FULL acc.
50	0.126	0.088	0.027	0.241	1.058	2.98	100.00 ± 0.00	100.00 ± 0.00	100.00 ± 0.00
100	0.146	0.086	0.027	0.260	1.061	2.75	100.00 ± 0.00	100.00 ± 0.00	100.00 ± 0.00
200	0.209	0.102	0.026	0.338	1.517	3.01	77.75 ± 30.05	87.45 ± 15.91	100.00 ± 0.00
500	0.744	0.106	0.026	0.877	2.605	2.25	66.90 ± 3.96	85.90 ± 15.56	100.00 ± 0.00
1000	1.499	0.048	0.026	1.574	5.328	2.60	54.60 ± 8.20	64.25 ± 2.47	84.35 ± 10.39

TABLE VI
CIFAR-10, REDUCED SCALE (10K TRAINING IMAGES, 10 EPOCHS): TEST ACCURACY (%), MEAN ± STD OVER 3 SEEDS.

Ratio	k	CRAIG	CRAIG-unw.	RANDOM
10%	1000	37.00 ± 1.58	35.00 ± 1.39	36.13 ± 1.71
20%	2000	43.19 ± 0.14	43.38 ± 1.25	44.01 ± 0.48
30%	3000	45.16 ± 2.35	47.46 ± 1.34	47.55 ± 0.91
50%	5000	50.14 ± 0.83	51.52 ± 0.65	51.68 ± 2.34
100% (FULL)	10000	59.02 ± 0.17		

TABLE VII
CIFAR-10 (REDUCED): RELATIVE GRADIENT ERROR (MEAN ± STD), CRAIG GRADIENT TIME, T_{sel}, T_{sub} (S), AND CRAIG END-TO-END SPEEDUP.

	Rel. grad. error					
Ratio	CRAIG	RANDOM	T_{grad}	T_{sel}	T_{sub}	E2E
10%	0.207 ± 0.031	0.802 ± 0.212	0.72	1.01	1.59	5.07
20%	0.164 ± 0.023	0.533 ± 0.110	0.71	1.01	3.22	3.11
30%	0.137 ± 0.023	0.333 ± 0.032	0.66	0.98	3.17	3.17
50%	0.095 ± 0.016	0.254 ± 0.045	0.62	0.97	6.56	1.75
FULL	–	–	–	–	13.17 ± 3.90	1.00

Gradient matching is necessary but not sufficient. On all three datasets, CRAIG achieves a lower gradient error than random sampling at the selection parameters. The improvement is modest on MNIST and large on CIFAR-10 and the time series. Accuracy follows only on MNIST, where the model is convex and the gradient geometry is stable during training. For the CNNs, a single selection at random initialization matches the gradients of an untrained network. This is a likely reason that the large gradient-error advantage on CIFAR-10 did not improve accuracy beyond the 10% ratio.

When does CRAIG pay off in time? CRAIG pays off only when $T_{\text{sel}} < T_{\text{full}} - T_{\text{sub}}$. In our runs, this inequality is governed by three factors:

1) *The greedy step dominates.* On MNIST, the pure-Python lazy greedy takes 77–81% of T_{sel}. Per-sample gradients

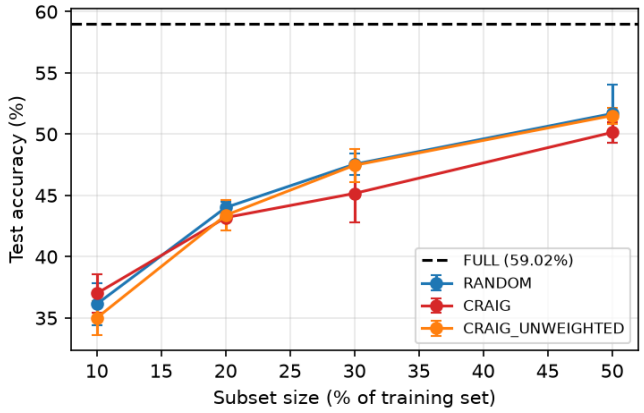


Fig. 6. CIFAR-10 (reduced scale): test accuracy vs. subset size (mean ± std, 3 seeds). The dashed line is FULL (59.02%).

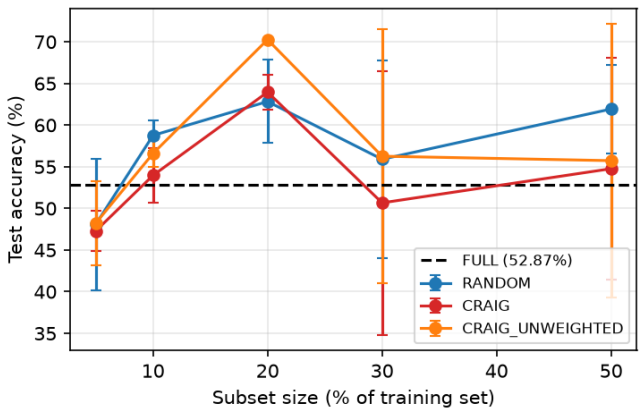


Fig. 7. Synthetic time series, hard setting (σ=2.0): test accuracy vs. subset size (mean ± std, 3 seeds). The large error bars and the low FULL accuracy (52.87%) show that this setting is not reliable for ranking methods.

TABLE VIII
SYNTHETIC TIME SERIES, HARD SETTING (σ=2.0, N=4000, L=100, 3 SEEDS). TOP: TEST ACCURACY (%). BOTTOM: RELATIVE GRADIENT ERROR, T_{sel}, T_{sub} (S), AND CRAIG END-TO-END SPEEDUP.

Ratio	k	CRAIG	CRAIG-unw.	RANDOM
5%	200	47.30 ± 2.39	48.23 ± 5.04	48.10 ± 7.90
10%	400	53.97 ± 3.30	56.60 ± 1.57	58.77 ± 1.80
20%	800	63.97 ± 2.08	70.27 ± 0.23	62.83 ± 4.99
30%	1200	50.67 ± 15.86	56.27 ± 15.25	55.90 ± 11.91
50%	2000	54.77 ± 13.33	55.73 ± 16.45	61.93 ± 5.33
100% (FULL)	4000		52.87 ± 5.90	

	Rel. grad. error				
Ratio	CRAIG	RANDOM	T_{sel}	T_{sub}	E2E
5%	0.093 ± 0.008	1.179 ± 0.458	0.73	0.13	2.92
10%	0.070 ± 0.005	0.662 ± 0.027	0.72	0.24	2.62
20%	0.055 ± 0.004	0.461 ± 0.050	0.67	0.47	2.20
30%	0.046 ± 0.003	0.490 ± 0.065	0.69	0.67	1.85
50%	0.034 ± 0.003	0.313 ± 0.186	0.71	1.11	1.38
FULL	–	–	–	2.52 ± 0.56	1.00

via `vmap` cost only about 1 s for 60k samples. Engineering effort should therefore target the greedy step, for example a compiled or vectorized implementation or a stochastic variant, rather than the gradient computation.

2) *Per-class size matters more than total size.* Selection cost scales super-linearly in the per-class sample count, whereas full training scales linearly in N. In the dataset-size study, CRAIG is faster end-to-end up to N=5,000 and slower from N=10,000. On full MNIST (about 6,000 samples per class), selection alone takes 4.47–5.80× the full training time.
3) *Cheap training makes break-even hard.* Full MNIST logistic regression trains in 6.58 s on 8 CPU cores, which leaves very little time to save. The selection cost does not depend on the number of epochs, while $T_{\text{full}} - T_{\text{sub}}$ grows with it, so longer training or more expensive models move the balance toward CRAIG. As a purely arithmetic illustration (not measured), the MNIST selection would have to be reused across about $T_{\text{sel}}/(T_{\text{full}} - T_{\text{sub}})$ training runs of the same length to amortize: $29.41/6.12 \approx 4.8$ runs at 5% and $38.20/2.43 \approx 15.7$ runs at 50%. Such reuse is valid only if the subset stays useful across runs.

In the CIFAR-10 and time-series settings, CRAIG does break even, because the gradient dimension or the per-class count is small. In those settings, however, uniform random subsets were at least as accurate and even cheaper, so the time saving alone is not an argument for CRAIG.

VIII. LIMITATIONS AND THREATS TO VALIDITY

- **CPU-only, non-isolated hardware.** All runs used a shared 8-core CPU machine. Timing noise is visible: selection-time std is up to 7.17 s on MNIST, and full-training-time std is 3.90 s on CIFAR-10. GPU and MPS paths are untested. The relative cost of the stages, and therefore the break-even point, will differ on accelerators.
- **Single, one-shot selection.** The subset is selected once from exact gradients at the initial weights. The original method re-selects during training for deep networks and uses feature-space bounds for convex models [1]. Our deep-network results should therefore not be read as a test of the original per-epoch protocol.
- **Simplified weighted training.** Our per-minibatch normalized weighted loss is a simplification of the weighted incremental-gradient updates in the original method.
- **Reduced CIFAR-10.** We used 10k of the 50k training images, a small CNN, 10 epochs, no augmentation, and 3 seeds. FULL reaches only 59.02%. These results are not comparable to published CIFAR-10 numbers.
- **Synthetic time series.** We used no real time-series benchmark. The easy setting saturates at 100%, and the hard setting is unstable, with FULL below several subsets. Neither supports conclusions about accuracy on real time-series data.
- **Few seeds in places.** We used 3 seeds for CIFAR-10 and the time series and 2 for the scaling studies, which

limits the statistical power of small accuracy differences. We report paired differences and win counts rather than significance tests.

- **Unoptimized greedy.** The lazy greedy is written in Python with a binary heap. A faster implementation would shift the break-even point. We did not implement stochastic or distributed greedy variants.

IX. Conclusion

We re-implemented CRAIG with exact vectorized per-sample gradients, blockwise similarity, lazy greedy facility location, and Voronoi weights, and evaluated it end-to-end on a CPU. On MNIST with logistic regression, the statistical claim reproduces: weighted CRAIG subsets have higher mean accuracy than random subsets of the same size at every ratio, and the weights are essential. The computational claim does not reproduce in our setting: selection costs 4.47–5.80$\times$ the full training time, giving an end-to-end speedup of only 0.16–0.22$\times$. The scaling study puts the break-even point between 5k and 10k training samples. With one-shot selection at initialization, CRAIG gave no reliable accuracy gain on reduced CIFAR-10 or on synthetic time series, despite much lower gradient approximation error. Natural next steps are a faster greedy implementation, per-epoch re-selection for deep networks, GPU timing, the full CIFAR-10 dataset, and real time-series benchmarks.

References

[1] B. Mirzasoleiman, J. Bilmes, and J. Leskovec, "Coresets for data-efficient training of machine learning models," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020.

[2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024–8035.

[3] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.

[4] M. Minoux, "Accelerated greedy algorithms for maximizing submodular set functions," in *Optimization Techniques*, ser. Lecture Notes in Control and Information Sciences. Springer, 1978, vol. 7, pp. 234–243.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[6] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.